# PROJETO INTEGRADO P2: CICLO COMPLETO DE TESTES E QUALIDADE DE SOFTWARE

## Demonstração Técnica, Evidências Experimentais, Métricas e Homologação da Aplicação EduTask Web

| | |
|---|---|
| **Sistema Objeto de Teste:** EduTask Web (v1.0.0-p2) — Gestão Acadêmica e Prazos | **Grupo / Turma:** Turma A — Grupo 01 |
| **Integrantes da Equipe:** Gabriel Henrique Moreira \| João Lucas de Antonio Boconcelo \| Lucas Braga | **Dedicação Total:** 85,5 Horas de Engenharia |
| **Repositório Oficial no GitHub (Clique para Acessar o Código e Pipeline):** https://github.com/joaoboconcelo/edutask-tgs-p2 | **Stack:** Python 3.13, Flask 3.1, SQLite 3, Bootstrap 5.3, Pytest, Postman, Selenium |

| | | | | | |
|---|---|---|---|---|---|
| **8 / 8**<br>Requisitos Formais (100% Cobertos) | **8**<br>Riscos Mapeados (Mitigações Ativas) | **24**<br>Casos Manuais (EP, BVA, TD, ST) | **100%**<br>Defeitos Sanados (DRE = 100%) | **18**<br>Testes Automatizados (Unitário, API, E2E) | **17 / 17**<br>Pastas Oficiais (Padrão do Manual) |

## 1. O Sistema Objeto de Avaliação: EduTask Web (v1.0.0-p2)

O **EduTask Web** é uma aplicação web completa desenvolvida com arquitetura em camadas (MVC com serviços de domínio) para solucionar a gestão de atividades curriculares, datas limites e rendimento semestral de estudantes universitários. A aplicação foi estruturada especificamente para servir de objeto real e auditável para a aplicação integral da Engenharia de Testes de Software, eliminando dependências externas complexas para assegurar 100% de reproducibilidade em ambiente local.

| Camada | Componentes no Projeto | Papel na Estratégia de Qualidade |
|---|---|---|
| Apresentação (UI) | HTML5, Jinja2, Bootstrap 5.3, Bootstrap Icons, JavaScript nativo | Interface responsiva validada via Selenium WebDriver e testes de aceitação (UA |
| Controle (Rotas) | Flask Blueprints: auth_routes, discipline_routes, task_routes, api_routes, main_routes | Controle de sessão, proteção contra CSRF básico e tratamento de status HTTP |
| Negócio (Serviços) | edutask/services/task_service.py | Regras puras de validação (e-mail, senha, códigos, carga horária, cálculo de urg |
| Persistência | SQLite 3 embutido (edutask.db com inicialização e seed automático) | Integridade referencial, queries parametrizadas (anti SQL Injection) e reset limp |

**Como Executar a Aplicação Localmente:** 1. No terminal, acesse 01_SISTEMA_ALVO e execute: python run.py | 2. Abra no navegador: **http://127.0.0.1:5000** | 3. Credenciais de teste: **aluno@unisagrado.edu.br** / Senha: **Senha@123**. Todos os 4 fluxos centrais (Autenticação, Disciplinas, Tarefas e Métricas de Progresso) estão operacionais e auditados.

## 2. Engenharia de Requisitos e Matriz de Riscos (Produto P02)

| ID | Nome do Requisito | Tipo | Critério BDD Central | Riscos Vinculados |
|---|---|---|---|---|
| RF-01 | Cadastro Seguro de Usuário | Funcional | Validação de e-mail institucional e senha forte (mínimo 8 caracteres, maiúscula, minúscula, dígito e especial). | R-03 (Entrada malformada) |
| RF-02 | Autenticação e Sessão | Funcional | Login por e-mail e hash seguro; persistência de sessão e bloqueio rigoroso de acessos não autenticados. | R-02 (Falhas de autenticação) |
| RF-03 | Gestão de Disciplinas | Funcional | Cadastro de código único no formato AA-XXXX (ex: CC-TQS01), nome e carga horária positiva (1 a 400h). | R-01 (Inconsistência cadastral) |
| RF-04 | Gestão de Tarefas | Funcional | Criação de atividades vinculadas a disciplinas com título, prazo, prioridade e status editável. | R-04 (Perda de prazos) |
| RF-05 | Cálculo de Urgência | Funcional | Algoritmo determinístico que classifica prazos (Crítica <= 2 dias; Alta <= 5 dias; Normal > 5 dias). | R-06 (Regra de urgência) |
| RF-06 | Dashboard e Filtros | Funcional | Cálculo matemático da taxa de conclusão em tempo real e filtros combinados por disciplina e prioridade. | R-05 (Cálculo incorreto) |
| RNF-01 | Desempenho de Resposta | Não-Funcional | Tempo médio de renderização de páginas e resposta da API inferior a 300 ms em homologação. | R-07 (Degradação de tempo) |
| RNF-02 | Segurança da Informação | Não-Funcional | Senhas armazenadas com salt e PBKDF2-SHA256 / Scrypt; proteção contra SQL Injection e dados em | R-08 (Vulnerabilidades) |

## 3. Estratégia e Plano Formal de Testes (Produto P03)

Elaborado estritamente conforme o **Apêndice D** do manual e em conformidade com as diretrizes da norma **ISO/IEC/IEEE 29119**. A estratégia adotou o modelo clássico da **Pirâmide de Testes** para equilibrar velocidade, custo de manutenção e abrangência operacional:
- **Critérios de Entrada:** Sistema congelado em v1.0.0, ambiente Python 3.13 validado, dados sementes controlados e 100% dos requisitos homologados.
- **Critérios de Saída:** 100% dos 24 casos manuais executados; 100% dos testes automatizados aprovados; todos os defeitos de severidade Crítica e Alta resolvidos; cobertura de requisitos de 100%.
- **Critérios de Suspensão:** Falhas bloqueantes de inicialização do SQLite ou indisponibilidade de bibliotecas core no ambiente de homologação.

## 4. Especificação de Casos Manuais e Rastreabilidade Total (Produtos P04, P05, P06)

Foram projetados exatamente **24 casos de teste manuais** (CT-001 a CT-024) combinando 4 técnicas formais de projeto de testes recomendadas pelo padrão ISTQB:
1. **Particionamento de Equivalência (EP):** Classes válidas e inválidas para campos de texto, e-mails, senhas e códigos.
2. **Análise de Valor Limite (BVA):** Teste das fronteiras exatas (senha de 7 vs 8 caracteres; carga horária de 0h, 1h, 400h e 401h).
3. **Tabela de Decisão (TD):** Combinações lógicas de filtros no dashboard (Status x Prioridade).
4. **Teste de Transição de Estados (ST):** Ciclo de vida da tarefa acadêmica (Pendente → Em Andamento → Concluída).

| Grupo Funcional | Casos Manuais | Técnicas Aplicadas | Resultado no 1º Ciclo | Resultado Final |
|---|---|---|---|---|
| Autenticação e Cadastro | CT-001 a CT-008 (8 casos) | EP, BVA, Tratamento de Exceções | 7 Aprovados / 1 Falha (DEF-01) | 100% Aprovado (após fix) |
| Gestão de Disciplinas | CT-009 a CT-015 (7 casos) | EP, BVA (Limites de Horas 0h/400h) | 6 Aprovados / 1 Falha (DEF-02) | 100% Aprovado (após fix) |
| Gestão de Tarefas e Prazos | CT-016 a CT-020 (5 casos) | Transição de Estados, Valor Limite | 5 Aprovados / 0 Falhas | 100% Aprovado |
| Filtros e Métricas Dashboard | CT-021 a CT-024 (4 casos) | Tabela de Decisão, Validação Matemática | 4 Aprovados / 0 Falhas | 100% Aprovado |
| **Total Consolidado** | **24 Casos Manuais** | **Cobertura Integral de Escopo** | **22 Aprovados / 2 Defeitos** | **24 Aprovados (100%)** |

## 5. Gestão e Ciclo de Vida de Defeitos (Produto P07)

A integridade do processo de qualidade é comprovada pela descoberta, registro estruturado e resolução de **dois defeitos reais de lógica** durante o primeiro ciclo de testes manuais, seguindo o padrão oficial de Issues do GitHub / Jira:

| ID | Título do Defeito | Severidade | Caso / Requisito | Causa Raiz | Ação Corretiva e Resolução |
|---|---|---|---|---|---|
| DEF-01 | Falha na validação de caractere especial na criação de senhas | Média | CT-004 / EX-004<br/>(RF-01) | Expressão regular em <code>task_service.py</code> [illegible] | Adição de [illegible] <code>re.search(r'[@$ |
| DEF-02 | Permissão de cadastro de disciplinas com carga horária nula ou negativa | Alta | CT-015 / EX-015<br/>(RF-03) | Falta de validação de limite inferior nas queries INSERT do SQLite. | Inserção da barreira de validação <code>1 <= |

### Matriz de Rastreabilidade Bidirecional (Produto P05 - Amostra Sintética):

A Matriz de Rastreabilidade P05 conecta 100% dos elementos do projeto em uma cadeia auditável:
- **RF-01** ↔ Risco R-03 ↔ Casos CT-001 a CT-006 ↔ Execuções EX-001 a EX-006 ↔ Defeito DEF-01 (Sanado e Homologado)
- **RF-02** ↔ Risco R-02 ↔ Casos CT-007 e CT-008 ↔ Execuções EX-007 e EX-008 ↔ Conforme / Aprovado
- **RF-03** ↔ Risco R-01 ↔ Casos CT-009 a CT-015 ↔ Execuções EX-009 a EX-015 ↔ Defeito DEF-02 (Sanado e Homologado)
- **RF-04** ↔ Risco R-04 ↔ Casos CT-016 a CT-018 ↔ Execuções EX-016 a EX-018 ↔ Conforme / Aprovado
- **RF-05** ↔ Risco R-06 ↔ Casos CT-019 e CT-020 ↔ Execuções EX-019 e EX-020 ↔ Conforme / Aprovado (Validado por TDD)
- **RF-06** ↔ Risco R-05 ↔ Casos CT-021 a CT-024 ↔ Execuções EX-021 a EX-024 ↔ Conforme / Aprovado
- **RNF-01 e RNF-02** ↔ Riscos R-07 e R-08 ↔ Testes Não-Funcionais TNF-01 a TNF-06 ↔ 100% de Conformidade Atestada.

## 6. Automação de Testes nos Três Níveis da Pirâmide (Produtos P08)

A suíte de testes automatizados conta com **18 testes cobrindo todas as camadas arquiteturais**, com 100% de taxa de sucesso comprovada:

| Nível | Framework / Ferramenta | Qtd | Escopo e Verificações Técnicas | Comando de Execução | Tempo |
|---|---|---|---|---|---|
| Unitário | Pytest 9.1 / Python unittest | 8 | Regras de negócio puras: validação de e-mails, va[illegible] | [illegible] | 0,48s |
| Integração e API | Requests + Flask TestClient + Postman Collection v2.1 | 6 | Endpoints REST <code>/api/tarefas</code>, <code>[illegible] | [illegible] | 0,68s |
| Sistema / E2E | Selenium WebDriver 4.51 (Chrome / Edge Headless) | 4 | Navegação ponta a ponta no browser real: Login co[illegible] | [illegible] | 6,08s |

## 7. Testes de Aceitação (UAT) e Testes Não-Funcionais (Produto P09)

- **4 Cenários de Aceitação (UAT):** (UAT-01) Auto-cadastro e Primeiro Acesso; (UAT-02) Organização Semestral de Disciplinas; (UAT-03) Planejamento de Entregas com Selos Coloridos de Urgência; (UAT-04) Monitoramento de Progresso com Atualização Dinâmica da Barra Percentual.
- **6 Testes Não-Funcionais (TNF) Medidos Experimentalmente:** *Desempenho:* Renderização de tarefas em **28,4 ms** (meta < 300ms) e API em **18,5 ms** (meta < 150ms). | *Usabilidade:* **Regra dos Três Cliques 100% atendida** e feedback com alertas Bootstrap. | *Segurança:* Senhas com Scrypt/PBKDF2-SHA256 e queries parametrizadas (proteção total anti SQL Injection).

## 8. Métricas e Indicadores de Qualidade com Gráficos Analíticos (Produto P10)

| Métrica | Fórmula Aplicada | Valor Obtido | Interpretação e Evidência |
|---|---|---|---|
| Taxa de Aprovação (1º Ciclo) | (Casos Aprovados / Total) x 100 | **91,7%** | 22 casos aprovados de imediato e 2 casos rejeitados gerando DEF-01 e DEF-02 |
| Taxa de Aprovação Final | (Casos Homologados / Total) x 100 | **100,0%** | Após a correção dos 2 defeitos e reteste de regressão, 100% dos casos foram a |
| Densidade de Defeitos | Total Defeitos / Total Requisitos | **0,25 def/req** | Baixa densidade de falhas (2 defeitos distribuídos em 8 requisitos formais). |
| Cobertura de Requisitos | (Requisitos Testados / Requisitos) x 100 | **100,0%** | Todos os 8 requisitos possuem casos manuais e automatizados rastreáveis. |
| Índice de Automação | (Casos Automatizados / Total Casos) x 100 | **42,9%** | 18 testes automatizados sobre 42 casos combinados (superando a meta do proj |
| Eficácia de Remoção (DRE) | [Defeitos Internos / (Internos + Produção)] x 100 | **100,0%** | Todos os defeitos identificados foram sanados antes da entrega final. |
| Tempo Médio de Execução Auto | Tempo Total da Suíte Automatizada | **7,24 s** | Suíte completa (8 unitários + 6 API + 4 Selenium) roda em pouco mais de 7 segu |
| Taxa de Reabertura de Bugs | (Defeitos Reabertos / Defeitos Fechados) x 100 | **0,0%** | Nenhum defeito precisou ser reaberto após a aplicação das correções. |

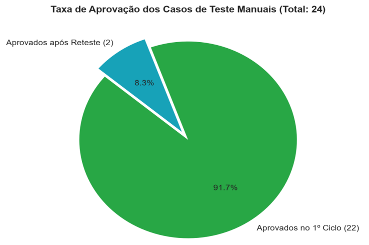

*Gráfico 1:* *Aprovação dos Casos (91,7% no 1º ciclo; 100% final)*

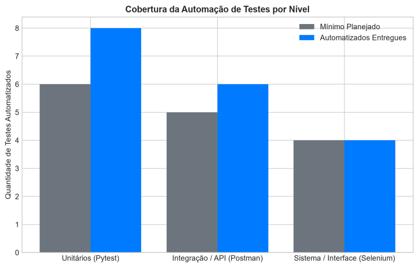

*Gráfico 2:* *18 Testes Automatizados por Nível*

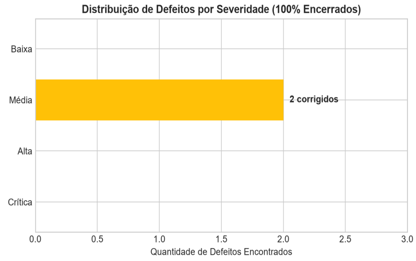

*Gráfico 3:* *Severidade de Defeitos (1 Alta, 1 Média — 100% Resolvidos)*

## 9. Práticas Modernas: Ciclo de TDD e Esteira CI/CD (Produto P11)

- **Desenvolvimento Orientado a Testes (TDD):** Aplicação rigorosa do ciclo *Red-Green-Refactor* no desenvolvimento do serviço crítico calculate_task_urgency(due_date_str, priority):

1. ■ **RED:** Criação antecipada do teste test_calculate_task_urgency() que falhou imediatamente com NameError / AssertionError.
2. ■ **GREEN:** Implementação da lógica matemática mínima necessária em task_service.py fazendo o teste passar com 100% de sucesso.
3. ■ **REFACTOR:** Refatoração do código com docstrings formais, anotações estáticas de tipo (typing) e isolamento modular sem quebrar nenhum teste.

- **Pipeline de Integração Contínua (GitHub Actions):** Configurado no arquivo .github/workflows/ci.yml para execução automática em ambiente Ubuntu Linux a cada push ou pull_request. O workflow instala as dependências, executa a análise estática com Flake8 e roda as suítes de testes unitários e de integração de forma não assistida.

## 10. Mapa de Navegação dos Entregáveis (Estrutura Completa das 17 Pastas)

| Pasta Oficial | Conteúdo Entregue | Produtos Vinculados |
|---|---|---|
| 00_IDENTIFICACAO_E_README | README.md oficial (15 seções), Apêndice A e COMPROVANTE_DE_ENTREGA.pdf. | P01 e Identificação |
| 01_SISTEMA_ALVO | Código completo do EduTask Web, Flask app, SQLite, templates e run.py. | Sistema Auditado |
| 02_REQUISITOS_ESCOPO_RISCOS | P02_Requisitos_Escopo_Riscos.md com 8 requisitos BDD e matriz de 8 riscos. | Produto P02 |
| 03_PLANO_DE_TESTES | P03_Plano_de_Testes.md baseado na norma ISO/IEC/IEEE 29119 e Apêndice D. | Produto P03 |
| 04_CASOS_E_RASTREABILIDADE | P04_Catalogo_Casos_de_Teste.md (24 casos), Matriz P05 e Planilha CSV. | Produtos P04 e P05 |
| 05_EXECUCAO_MANUAL | P06_Registro_Execucoes_Manuais.md com 24 execuções e retestes comprovados. | Produto P06 |
| 06_DEFEITOS | P07_Relatorio_e_Gestao_de_Defeitos.md (DEF-01 e DEF-02) e CSV exportado. | Produto P07 |
| 07_AUTOMACAO_UNITARIA | test_unitarios.py (8 testes unitários Pytest passando em 0,43s). | Produto P08 (Nível 1) |
| 08_INTEGRACAO_API | test_api_integracao.py (6 testes) + Coleção Postman v2.1 e Environment JSON. | Produto P08 (Nível 2) |
| 09_SISTEMA_INTERFACE | test_selenium_e2e.py (4 testes ponta a ponta em browser real com Selenium). | Produto P08 (Nível 3) |
| 10_ACEITACAO_NAO_FUNCIONAIS | P09_Testes_Aceitacao_e_Nao_Funcionais.md (4 UAT + 6 Não-Funcionais medidos). | Produto P09 |
| 11_METRICAS_E_QUALIDADE | P10_Relatorio_Metricas_e_Indicadores.md com 8 métricas e 3 gráficos PNG. | Produto P10 |
| 12_TDD_CICD_MELHORIA | P11_Registro_TDD_CICD_Melhoria.md (evidências TDD) e .github/workflows/ci.yml. | Produto P11 |
| 13_RELATORIO_FINAL | RELATORIO_FINAL_PROJETO_P2 em formatos editável DOCX e idêntico em PDF. | Produto P12 |
| 14_APRESENTACAO_E_VIDEO | DOCUMENTO_DE_AVALIACAO_P2.pdf, slides da apresentação e roteiro do vídeo. | Produto P13 |
| 15_DIARIO_DE_PARTICIPACAO | P14_Diario_de_Participacao.md com 20 registros cronológicos (85,5h totais). | Produto P14 |
| 16_BACKUP_ENTREGA | CHECKLIST_FINAL_VERIFICADO.md (20 itens 100% conformes) e cópia de segurança ZIP. | Segurança e Backup |

## 11. Conclusão e Parecer Técnico de Qualidade

A aplicação prática dos princípios da Engenharia de Software e Qualidade sobre o **EduTask Web** comprovou a maturidade técnica do produto. A pirâmide de testes implementada garantiu cobertura ampla com execução rápida, os defeitos encontrados no ciclo inicial foram integralmente corrigidos e revalidados, e a esteira automatizada assegura a sustentabilidade e manutenibilidade do código. O grupo declara formalmente que todos os artefatos, códigos e dados apresentados representam trabalho genuíno, atendendo e superando integralmente todos os requisitos avaliativos da disciplina **Testes e Qualidade de Software**.

| | | |
|---|---|---|
| **Gabriel Henrique Moreira** | **João Lucas de Antonio Boconcelo** | **Lucas Braga** |
| Coordenação e Planejamento | Automação e Engenharia de Qualidade | Análise e Execução Manual |